Music recommendation

```
# Install dependencies (Colab-safe)

%pip install -q gdown kaleido plotly
```

```
!gdown 1733JgZmGePOT-JWX_IHN7OQdGuHoHeuo
```

```
Downloading...
From: https://drive.google.com/uc?id=1733JgZmGePOT-JWX_IHN7OQdGuHoHeuo
To: /content/user_listening_history_rows.csv
100% 3.61M/3.61M [00:00<00:00, 106MB/s]
```

```
!gdown 1RJaWJbtT4GUp4FmIsL9Wj1jRd48HC6fT
```

```
Downloading...
From: https://drive.google.com/uc?id=1RJaWJbtT4GUp4FmIsL9Wj1jRd48HC6fT
To: /content/Music Info.csv.zip
100% 5.93M/5.93M [00:00<00:00, 50.9MB/s]
```

```
!gdown 1sgNxSpangDMdBbZThXo0QWm3JMtGJgfn
```

```
Downloading...
From (original): https://drive.google.com/uc?id=1sgNxSpangDMdBbZThXo0QWm3JMtGJgfn
From (redirected): https://drive.google.com/uc?id=1sgNxSpangDMdBbZThXo0QWm3JMtGJgfn&confirm=t&uuid=e5a4cb3e-73f3-4f82-861e-ecdfaae64c34
To: /content/User Listening History.csv.zip
100% 126M/126M [00:03<00:00, 38.1MB/s]
```

```
!unzip 'User Listening History.csv.zip'
```

```
Archive:  User Listening History.csv.zip
replace User Listening History.csv? [y]es, [n]o, [A]ll, [N]one, [r]ename: A
  inflating: User Listening History.csv
```

```
!unzip "Music Info.csv.zip"
```

```
Archive:  Music Info.csv.zip
replace Music Info.csv? [y]es, [n]o, [A]ll, [N]one, [r]ename: A
  inflating: Music Info.csv
```

```
import pandas as pd


mi = pd.read_csv('User Listening History.csv')
mi.head()
```

	track_id	user_id	playcount
0	TRIRLYL128F42539D1	b80344d063b5ccb3212f76538f3d9e43d87dca9e	1
1	TRFUPBA128F934F7E1	b80344d063b5ccb3212f76538f3d9e43d87dca9e	1
2	TRLQPQJ128F42AA94F	b80344d063b5ccb3212f76538f3d9e43d87dca9e	1
3	TRTUCUY128F92E1D24	b80344d063b5ccb3212f76538f3d9e43d87dca9e	1
4	TRHDDQG12903CB53EE	b80344d063b5ccb3212f76538f3d9e43d87dca9e	1

```
import zipfile
```

```
from pathlib import Path

import gdown

# --- Colab runtime toggles ---
QUICK_RUN = True  # False = score all unseen items per user in ranking (very slow)
RANKING_N_USERS = 25 if QUICK_RUN else 150
RANKING_MAX_CANDIDATES = 400 if QUICK_RUN else None  # None = full candidate pool

# Download ML-100K via gdown (official GroupLens HTTPS URL is supported by gdown)
DATA_ROOT = Path("data")
ML100K_PATH = DATA_ROOT / "ml-100k"
ZIP_PATH = Path("ml-100k.zip")
ML100K_URL = "https://files.grouplens.org/datasets/movielens/ml-100k.zip"

if not (ML100K_PATH / "u.data").exists():
    DATA_ROOT.mkdir(parents=True, exist_ok=True)
    gdown.download(ML100K_URL, str(ZIP_PATH), quiet=False)
    with zipfile.ZipFile(ZIP_PATH, "r") as zf:
        zf.extractall(DATA_ROOT)
    assert (ML100K_PATH / "u.data").exists(), "Expected data/ml-100k/u.data after unzip"
else:
    print("Reusing existing dataset at", ML100K_PATH)
```

```
Reusing existing dataset at data/ml-100k
```

```
import random
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import torch
import torch.nn as nn
import torch.nn.functional as F
from plotly.subplots import make_subplots
from sklearn.decomposition import TruncatedSVD
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.model_selection import train_test_split

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", None)

path = Path("data/ml-100k")
images_dir = Path("images")
images_dir.mkdir(parents=True, exist_ok=True)


def save_plotly_figure(fig, filename_stem):
    """Save Plotly figure to images/ as PNG (requires kaleido)."""
    png_path = images_dir / f"{filename_stem}.png"
    try:
        fig.write_image(str(png_path))
        print(f"Saved plot: {png_path}")
    except Exception as exc:
        raise RuntimeError(
            "Failed to save Plotly figure as PNG. Install kaleido and rerun."
        ) from exc
```

1. Load and merge data

```
ulh_df = pd.read_csv('User Listening History.csv')
ulh_df.head()
```

	track_id	user_id	playcount
0	TRIRLYL128F42539D1	b80344d063b5ccb3212f76538f3d9e43d87dca9e	1
1	TRFUPBA128F934F7E1	b80344d063b5ccb3212f76538f3d9e43d87dca9e	1
2	TRLQPQJ128F42AA94F	b80344d063b5ccb3212f76538f3d9e43d87dca9e	1
3	TRTUCUY128F92E1D24	b80344d063b5ccb3212f76538f3d9e43d87dca9e	1
4	TRHDDQG12903CB53EE	b80344d063b5ccb3212f76538f3d9e43d87dca9e	1

```
ulh_df['liked'] = ulh_df['playcount'].apply(lambda x: 1 if x > 0 else 0)
```

2. Filter, train/test split, user–item matrix

We keep **heavy users and popular items** (≥50 ratings each) to stabilize similarity estimates. The training matrix uses **mean-centered** ratings per user for cosine similarity; missing entries are filled with zero only in the normalized matrix used for similarities.

```
user_counts = ulh_df.groupby("user_id")["track_id"].nunique()
movie_counts = ulh_df.groupby("track_id")["user_id"].nunique()
active_users = user_counts[user_counts >= 150].index
active_movies = movie_counts[movie_counts >= 1000].index
print(len(active_users))
print(len(active_movies))
# No filter yet

df_filtered = ulh_df[ulh_df.user_id.isin(active_users) & ulh_df.track_id.isin(active_movies)].copy()
print(df_filtered)
print(
    f"Filtered: {df_filtered['user_id'].nunique():,} users | "
    f"{df_filtered['track_id'].nunique():,} movies | "
    f"{len(df_filtered):,} ratings"
)

train_df, test_df = train_test_split(df_filtered, test_size=0.2, random_state=42)
print(f"Train: {len(train_df):,}  |  Test: {len(test_df):,}")


def build_user_item_matrix(data):
    """Pivot to user×movie with NaN for missing ratings."""
    return data.pivot_table(index="user_id", columns="track_id", values="playcount")


train_matrix = build_user_item_matrix(train_df)
print(
    f"User-Item matrix: {train_matrix.shape}  |  "
    f"Density: {train_matrix.notna().sum().sum() / train_matrix.size * 100:.2f}%"
)
user_means = train_matrix.mean(axis=1)
train_matrix_norm = train_matrix.subtract(user_means, axis=0).fillna(0)
```

```
803
2185
```

```
                  track_id                                 user_id  \
123      TRTSSUT128F1472A51  5a905f000fc1ff3df7ca807d57edb608863db05d
124      TRNJLKP128F427CE28  5a905f000fc1ff3df7ca807d57edb608863db05d
125      TRGAOLV128E0789D40  5a905f000fc1ff3df7ca807d57edb608863db05d
126      TREAQSX128E07818CA  5a905f000fc1ff3df7ca807d57edb608863db05d
127      TRUMDRI128F424FEFC  5a905f000fc1ff3df7ca807d57edb608863db05d
...                     ...                                      ...
9701040  TRCODGA128F92DD23A  491d048e26c51fcda0744355bf191d4ccf36f118
9701041  TRVKLGW12903CB4523  491d048e26c51fcda0744355bf191d4ccf36f118
9701042  TRUIIPM128F147CBD2  491d048e26c51fcda0744355bf191d4ccf36f118
9701044  TRVWQGQ128F933904F  491d048e26c51fcda0744355bf191d4ccf36f118
9701046  TREXCHD128EF33FB4E  491d048e26c51fcda0744355bf191d4ccf36f118

         playcount  liked
123              1      1
124              1      1
125              2      1
126              2      1
127              3      1
...            ...    ...
9701040          2      1
9701041          3      1
9701042          1      1
9701044          4      1
9701046          1      1

[74853 rows x 4 columns]
Filtered: 803 users | 2,172 movies | 74,853 ratings
Train: 59,882  |  Test: 14,971
User-Item matrix: (803, 2157)  |  Density: 3.46%
```

Experiments overview

2.1 User-based collaborative filtering (UBCF)

Predict a rating as the user's mean plus a **similarity-weighted** combination of mean-centered ratings from the top-N most similar users who rated that item. Similarity is **cosine** on the mean-centered user vectors.

```
user_sim_matrix = cosine_similarity(train_matrix_norm)
user_sim_df = pd.DataFrame(
    user_sim_matrix,
    index=train_matrix_norm.index,
    columns=train_matrix_norm.index,
)


def predict_user_based(user_id, movie_id, n_neighbors=20):
    """Top-N similar users who rated the movie; weighted mean-centered blend + user mean."""
    if user_id not in user_sim_df.index or movie_id not in train_matrix.columns:
        return np.nan
    rated_mask = train_matrix[movie_id].notna()
    rated_users = train_matrix[movie_id][rated_mask].index
    if len(rated_users) == 0:
        return np.nan
    sims = user_sim_df.loc[user_id, rated_users]
    top_sims = sims.nlargest(n_neighbors)
    numerator = (top_sims * train_matrix_norm.loc[top_sims.index, movie_id]).sum()
    denominator = top_sims.abs().sum()
    if denominator == 0:
        return user_means.get(user_id, 3.0)
    return user_means.get(user_id, 3.0) + numerator / denominator
```

```
test_sample = test_df.sample(min(2000, len(test_df)), random_state=42)

ubcf_preds, ubcf_actuals = [], []
for _, row in test_sample.iterrows():
    pred = predict_user_based(row["user_id"], row["track_id"])
    if not np.isnan(pred):
        ubcf_preds.append(np.clip(pred, 1, 5))
        ubcf_actuals.append(row["playcount"])

ubcf_rmse = np.sqrt(np.mean((np.array(ubcf_preds) - np.array(ubcf_actuals)) ** 2))
ubcf_mae = np.mean(np.abs(np.array(ubcf_preds) - np.array(ubcf_actuals)))
print(
    f"User-Based CF  →  RMSE: {ubcf_rmse:.4f}  |  MAE: {ubcf_mae:.4f}  "
    f"(coverage: {len(ubcf_preds)/len(test_sample)*100:.1f}%)"
)
```

```
User-Based CF  →  RMSE: 2.8491  |  MAE: 1.2910  (coverage: 99.9%)
```

2.2 Item-based collaborative filtering (IBCF)

Symmetric idea in **item space**: predict using items the user already rated that are most similar to the target movie (cosine on item vectors of mean-centered ratings).

```
item_sim_matrix = cosine_similarity(train_matrix_norm.T)
item_sim_df = pd.DataFrame(
    item_sim_matrix,
    index=train_matrix_norm.columns,
    columns=train_matrix_norm.columns,
)


def predict_item_based(user_id, movie_id, n_neighbors=20):
    if user_id not in train_matrix.index or movie_id not in item_sim_df.index:
        return np.nan
    user_ratings = train_matrix.loc[user_id].dropna()
    if len(user_ratings) == 0:
        return np.nan
    sims = item_sim_df.loc[movie_id, user_ratings.index]
    top_sims = sims.nlargest(n_neighbors)
    numerator = (top_sims * user_ratings[top_sims.index]).sum()
    denominator = top_sims.abs().sum()
    if denominator == 0:
        return user_means.get(user_id, 3.0)
    return numerator / denominator


ibcf_preds, ibcf_actuals = [], []
for _, row in test_sample.iterrows():
    pred = predict_item_based(row["user_id"], row["track_id"])
    if not np.isnan(pred):
        ibcf_preds.append(np.clip(pred, 1, 5))
        ibcf_actuals.append(row["playcount"])

ibcf_rmse = np.sqrt(np.mean((np.array(ibcf_preds) - np.array(ibcf_actuals)) ** 2))
ibcf_mae = np.mean(np.abs(np.array(ibcf_preds) - np.array(ibcf_actuals)))
print(
    f"Item-Based CF  →  RMSE: {ibcf_rmse:.4f}  |  MAE: {ibcf_mae:.4f}  "
    f"(coverage: {len(ibcf_preds)/len(test_sample)*100:.1f}%)"
)
```

```
Item-Based CF  →  RMSE: 2.7658  |  MAE: 1.1479  (coverage: 99.9%)
```

```
N_FACTORS = 50
n_components = min(N_FACTORS, min(train_matrix.shape) - 1)

svd_model = TruncatedSVD(n_components=n_components, random_state=42)
train_centered = train_matrix.subtract(user_means, axis=0).fillna(0)
user_factors = svd_model.fit_transform(train_centered)
item_factors = svd_model.components_.T
svd_recon_centered = user_factors @ item_factors.T
svd_pred_matrix = pd.DataFrame(
    svd_recon_centered,
    index=train_matrix.index,
    columns=train_matrix.columns,
).add(user_means, axis=0)


def predict_svd(user_id, movie_id):
    if user_id not in svd_pred_matrix.index or movie_id not in svd_pred_matrix.columns:
        return np.nan
    return svd_pred_matrix.loc[user_id, movie_id]


svd_preds, svd_actuals = [], []
for _, row in test_sample.iterrows():
    pred = predict_svd(row["user_id"], row["track_id"])
```

```
    if not np.isnan(pred):
        svd_preds.append(np.clip(pred, 1, 5))
        svd_actuals.append(row["playcount"])

svd_rmse = np.sqrt(np.mean((np.array(svd_preds) - np.array(svd_actuals)) ** 2))
svd_mae = np.mean(np.abs(np.array(svd_preds) - np.array(svd_actuals)))
print(
    f"SVD          →  RMSE: {svd_rmse:.4f}  |  MAE: {svd_mae:.4f}  "
    f"(coverage: {len(svd_preds)/len(test_sample)*100:.1f}%)"
)
```

```
SVD          →  RMSE: 2.8166  |  MAE: 1.2701  (coverage: 99.9%)
```

```
music_info_path = Path("Music Info.csv")
if not music_info_path.exists():
    raise FileNotFoundError(
        "Music Info.csv not found. Run the unzip cell for Music Info.csv.zip first."
    )
music_info_df = pd.read_csv(music_info_path)
rename_map = {}
for cand in ("track id", "TR_ID", "tid"):
    if cand in music_info_df.columns and "track_id" not in music_info_df.columns:
        rename_map[cand] = "track_id"
music_info_df = music_info_df.rename(columns=rename_map)
if "track_id" not in music_info_df.columns:
    raise ValueError(
        "Music Info.csv must contain a `track_id` column (or `track id` / TR_ID / tid). "
        f"Found columns: {list(music_info_df.columns)}"
    )
music_info_lookup = music_info_df.drop_duplicates(subset=["track_id"]).set_index("track_id")

def build_track_feature_tensor(track_ids_list, music_info_df, genre_col):
    """
    Side features per track: genre multi-hot + metadata presence flag.
    Rows follow track_ids_list order (train item index).
    """
    if genre_col is None:
        presence = np.array(
            [1.0 if tid in music_info_df.index else 0.0 for tid in track_ids_list],
            dtype=np.float32,
        )
        return torch.tensor(presence.reshape(-1, 1), dtype=torch.float32)

    all_genres = set()
    for tid in track_ids_list:
        if tid not in music_info_df.index:
            continue
        v = music_info_df.loc[tid][genre_col]
        if pd.notna(v):
            for g in str(v).replace(";", "|").replace(",", "|").split("|"):
                g = g.strip()
                if g:
                    all_genres.add(g)
    genre_list = sorted(all_genres)
    genre_to_idx = {g: i for i, g in enumerate(genre_list)}

    rows = []
    for tid in track_ids_list:
        vec = np.zeros(len(genre_list) + 1, dtype=np.float32)
        if tid in music_info_df.index:
            vec[-1] = 1.0
            v = music_info_df.loc[tid][genre_col]
            if pd.notna(v):
                for g in str(v).replace(";", "|").replace(",", "|").split("|"):
```

```
                    g = g.strip()
                    if g in genre_to_idx:
                        vec[genre_to_idx[g]] = 1.0
        rows.append(vec)
    return torch.tensor(np.stack(rows, axis=0), dtype=torch.float32)
```

```
def _pick_title_col(df):
    for c in df.columns:
        cl = c.lower()
        if cl in ("title", "track_name", "name", "song", "song_name"):
            return c
    return None


def _pick_artist_col(df):
    for c in df.columns:
        if "artist" in c.lower():
            return c
    return None


def _pick_genre_col(df):
    for c in df.columns:
        if "genre" in c.lower():
            return c
    return None


TITLE_COL = _pick_title_col(music_info_lookup)
ARTIST_COL = _pick_artist_col(music_info_lookup)
GENRE_COL = _pick_genre_col(music_info_lookup)


def track_label(track_id):
    if track_id not in music_info_lookup.index:
        return str(track_id)
    row = music_info_lookup.loc[track_id]
    parts = []
    if TITLE_COL is not None and pd.notna(row[TITLE_COL]):
        parts.append(str(row[TITLE_COL]))
    if ARTIST_COL is not None and pd.notna(row[ARTIST_COL]):
        parts.append(f"({row[ARTIST_COL]})")
    return " ".join(parts) if parts else str(track_id)


def track_genre_str(track_id):
    if GENRE_COL is None or track_id not in music_info_lookup.index:
        return "—"
    v = music_info_lookup.loc[track_id][GENRE_COL]
    return str(v) if pd.notna(v) else "—"
```

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

user_ids = train_matrix.index.tolist()
track_ids = train_matrix.columns.tolist()
user_to_idx = {u: i for i, u in enumerate(user_ids)}
track_to_idx = {t: i for i, t in enumerate(track_ids)}
track_ids_arr = np.array(track_ids)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
BPR_POS_MIN_PLAYCOUNT = 2

user_pos_items = (
    train_df[train_df["playcount"] >= BPR_POS_MIN_PLAYCOUNT]
    .groupby("user_id")["track_id"]
    .apply(lambda x: set(x.tolist()))
    .to_dict()
)
user_id_list = list(user_pos_items.keys())
print(
    f"BPR sampling: {len(user_id_list):,} / {train_df['user_id'].nunique():,} users "
    f"have at least one playcount>={BPR_POS_MIN_PLAYCOUNT} in train."
)


def build_user_aux_tensors(user_ids_list, train_data):
    """Per training user: log1p interaction count + mean playcount (standardized)."""
    interaction_counts = train_data.groupby("user_id").size()
    mean_playcount = train_data.groupby("user_id")["playcount"].mean()
    log_counts, mean_pc = [], []
    for uid in user_ids_list:
        log_counts.append(np.log1p(interaction_counts.get(uid, 0)))
        mean_pc.append(mean_playcount.get(uid, 0.0))
    log_counts = np.asarray(log_counts, dtype=np.float64)
    mean_pc = np.asarray(mean_pc, dtype=np.float64)
    log_counts = ((log_counts - log_counts.mean()) / (log_counts.std() + 1e-6)).astype(np.float32)
    mean_pc = ((mean_pc - mean_pc.mean()) / (mean_pc.std() + 1e-6)).astype(np.float32)
    return torch.tensor(np.stack([log_counts, mean_pc], axis=1), dtype=torch.float32)


def sample_bpr_batch_popular(batch_size=4096, mix_uniform=0.2):
    """BPR triplets with popularity^0.75 negatives mixed with uniform catalog draws."""
    u_idx, p_idx, n_idx = [], [], []
    for _ in range(batch_size):
        user_id = random.choice(user_id_list)
        pos_item = random.choice(tuple(user_pos_items[user_id]))
        seen = user_pos_items[user_id]
        neg_item = None
        for _attempt in range(30):
            if random.random() < mix_uniform:
                cand = random.choice(track_ids_arr)
            else:
                cand = track_ids[int(np.random.choice(len(track_ids), p=_item_pop_probs))]
            if cand not in seen:
                neg_item = cand
                break
        if neg_item is None:
            while True:
                cand = random.choice(track_ids_arr)
                if cand not in seen:
                    neg_item = cand
                    break
        u_idx.append(user_to_idx[user_id])
        p_idx.append(track_to_idx[pos_item])
        n_idx.append(track_to_idx[neg_item])
    return (
        torch.tensor(u_idx, dtype=torch.long),
        torch.tensor(p_idx, dtype=torch.long),
        torch.tensor(n_idx, dtype=torch.long),
    )


_train_track_idx_for_pop = train_df["track_id"].map(track_to_idx).to_numpy(dtype=np.int64)
```

```
_item_pop_counts = np.bincount(_train_track_idx_for_pop, minlength=len(track_ids)).astype(np.float64)
_item_pop_counts = np.maximum(_item_pop_counts, 1.0)
_item_pop_probs = _item_pop_counts**0.75
_item_pop_probs /= _item_pop_probs.sum()


class TwoTowerTrackBPR(nn.Module):
    """
    Two-tower + BPR for music: hybrid item tower (genre features + track ID embedding)
    and user tower (user ID embedding + activity aux stats). L2-normalized dot-product scores.
    """

    def __init__(
        self,
        n_users,
        n_items,
        track_features,
        user_aux,
        emb_dim=64,
        item_id_dim=32,
        hidden_dims=(128, 64),
    ):
        super().__init__()
        self.register_buffer("track_features", track_features)
        self.register_buffer("user_aux", user_aux)
        feat_dim = int(track_features.shape[1])
        self.user_emb = nn.Embedding(n_users, emb_dim)
        self.item_emb = nn.Embedding(n_items, item_id_dim)
        user_in = emb_dim + int(user_aux.shape[1])
        item_in = feat_dim + item_id_dim
        user_layers = []
        in_u = user_in
        for h in hidden_dims:
            user_layers.extend([nn.Linear(in_u, h), nn.ReLU(), nn.Dropout(0.1)])
            in_u = h
        self.user_proj = nn.Sequential(*user_layers)
        item_layers = []
        in_i = item_in
        for h in hidden_dims:
            item_layers.extend([nn.Linear(in_i, h), nn.ReLU(), nn.Dropout(0.1)])
            in_i = h
        self.item_proj = nn.Sequential(*item_layers)
        nn.init.normal_(self.user_emb.weight, std=0.01)
        nn.init.normal_(self.item_emb.weight, std=0.01)

    def encode_user(self, users):
        u = self.user_emb(users)
        a = self.user_aux[users]
        h = torch.cat([u, a], dim=1)
        return F.normalize(self.user_proj(h), p=2, dim=1)

    def encode_item(self, items):
        x = torch.cat([self.track_features[items], self.item_emb(items)], dim=1)
        return F.normalize(self.item_proj(x), p=2, dim=1)

    def score(self, users, items):
        return (self.encode_user(users) * self.encode_item(items)).sum(dim=1)


STEPS_PER_EPOCH = 120
BATCH_SIZE = 2048

track_feat_tensor = build_track_feature_tensor(track_ids, music_info_lookup, GENRE_COL).to(device)
user_aux_tensor = build_user_aux_tensors(user_ids, train_df)
```

```
two_tower_bpr = TwoTowerTrackBPR(
    n_users=len(user_ids),
    n_items=len(track_ids),
    track_features=track_feat_tensor,
    user_aux=user_aux_tensor.to(device),
    emb_dim=64,
    item_id_dim=32,
).to(device)
optimizer_two_tower = torch.optim.Adam(two_tower_bpr.parameters(), lr=1e-3, weight_decay=1e-6)

TWO_TOWER_EPOCHS = 10
TWO_TOWER_NEG_MIX_UNIFORM = 0.2
two_tower_bpr.train()
for epoch in range(1, TWO_TOWER_EPOCHS + 1):
    epoch_loss = 0.0
    for _ in range(STEPS_PER_EPOCH):
        users_b, pos_b, neg_b = sample_bpr_batch_popular(
            BATCH_SIZE, mix_uniform=TWO_TOWER_NEG_MIX_UNIFORM
        )
        users_b = users_b.to(device)
        pos_b = pos_b.to(device)
        neg_b = neg_b.to(device)
        pos_scores = two_tower_bpr.score(users_b, pos_b)
        neg_scores = two_tower_bpr.score(users_b, neg_b)
        loss = -F.logsigmoid(pos_scores - neg_scores).mean()
        optimizer_two_tower.zero_grad()
        loss.backward()
        optimizer_two_tower.step()
        epoch_loss += loss.item()
    print(f"Two-Tower BPR epoch {epoch}/{TWO_TOWER_EPOCHS} - loss: {epoch_loss/STEPS_PER_EPOCH:.4f}")

two_tower_bpr.eval()


@torch.no_grad()
def predict_two_tower_bpr(user_id, track_id):
    """Map dot score to [1, 5] for parity with other predictors."""
    if user_id not in user_to_idx or track_id not in track_to_idx:
        return np.nan
    u = torch.tensor([user_to_idx[user_id]], dtype=torch.long, device=device)
    i = torch.tensor([track_to_idx[track_id]], dtype=torch.long, device=device)
    score = two_tower_bpr.score(u, i).item()
    return float(1.0 + 4.0 * (1.0 / (1.0 + np.exp(-score))))


two_tower_preds, two_tower_actuals = [], []
for _, row in test_sample.iterrows():
    pred = predict_two_tower_bpr(row["user_id"], row["track_id"])
    if not np.isnan(pred):
        two_tower_preds.append(np.clip(pred, 1, 5))
        two_tower_actuals.append(row["playcount"])

two_tower_rmse = np.sqrt(np.mean((np.array(two_tower_preds) - np.array(two_tower_actuals)) ** 2))
two_tower_mae = np.mean(np.abs(np.array(two_tower_preds) - np.array(two_tower_actuals)))
print(
    f"Two-Tower BPR →  RMSE: {two_tower_rmse:.4f}  |  MAE: {two_tower_mae:.4f}  "
    f"(coverage: {len(two_tower_preds)/len(test_sample)*100:.1f}%)"
)
```

```
BPR sampling: 801 / 803 users have at least one playcount>=2 in train.
Two-Tower BPR epoch 1/10 - loss: 0.6319
Two-Tower BPR epoch 2/10 - loss: 0.5718
Two-Tower BPR epoch 3/10 - loss: 0.5593
Two-Tower BPR epoch 4/10 - loss: 0.5524
```

```
Two-Tower BPR epoch 5/10 - loss: 0.5466
Two-Tower BPR epoch 6/10 - loss: 0.5431
Two-Tower BPR epoch 7/10 - loss: 0.5413
Two-Tower BPR epoch 8/10 - loss: 0.5404
Two-Tower BPR epoch 9/10 - loss: 0.5391
Two-Tower BPR epoch 10/10 - loss: 0.5380
Two-Tower BPR →  RMSE: 3.2419  |  MAE: 2.2100  (coverage: 99.9%)
```

3. Ranking evaluation (Precision / Recall / NDCG @ K)

For each held-out user we take **relevant** test movies (rating ≥ `RELEVANCE_THRESHOLD`), rank **unseen** items by predicted score, and average **Precision@K**, **Recall@K**, and **NDCG@K**. With `QUICK_RUN`, only a random subset of candidates is ranked per user so Colab stays responsive; set `QUICK_RUN = False` for the exhaustive candidate pool.

```
RELEVANCE_THRESHOLD = 1
K_VALUES = [5, 10, 20]
all_tracks = train_matrix.columns.tolist()


def dcg(scores):
    return sum(s / np.log2(i + 2) for i, s in enumerate(scores))


def ndcg_at_k(recommended, relevant_set, k):
    hits = [1 if m in relevant_set else 0 for m in recommended[:k]]
    ideal = sorted(hits, reverse=True)
    return dcg(hits) / dcg(ideal) if dcg(ideal) > 0 else 0.0


def ranking_metrics(pred_func, test_data, k_values, n_users=200, max_candidates=None, rng_seed=42):
    """Precision@K, Recall@K, NDCG@K averaged over up to n_users eval users."""
    results = {k: {"precision": [], "recall": [], "ndcg": []} for k in k_values}
    rng = np.random.RandomState(rng_seed)

    test_users = test_data["user_id"].unique()
    train_users = set(train_matrix.index)
    eval_users = sorted(set(test_users) & train_users)[:n_users]

    for user_id in eval_users:
        user_test = test_data[test_data["user_id"] == user_id]
        relevant = set(user_test[user_test["playcount"] >= RELEVANCE_THRESHOLD]["track_id"])
        if len(relevant) == 0:
            continue

        seen = (
            set(train_matrix.loc[user_id].dropna().index)
            if user_id in train_matrix.index
            else set()
        )
        candidates = [m for m in all_tracks if m not in seen]
        if max_candidates is not None and len(candidates) > max_candidates:
            pick = rng.choice(len(candidates), size=max_candidates, replace=False)
            candidates = [candidates[i] for i in sorted(pick)]

        scores = [(m, pred_func(user_id, m)) for m in candidates]
        scores = [(m, s) for m, s in scores if not np.isnan(s)]
        scores.sort(key=lambda x: x[1], reverse=True)
        ranked = [m for m, _ in scores]

        for k in k_values:
            top_k = set(ranked[:k])
            hits = top_k & relevant
```

```
            results[k]["precision"].append(len(hits) / k)
            results[k]["recall"].append(len(hits) / len(relevant))
            results[k]["ndcg"].append(ndcg_at_k(ranked, relevant, k))

    return {
        str(k): {m: np.mean(v) for m, v in metrics.items()}
        for k, metrics in results.items()
    }


print("Computing ranking metrics …")
ubcf_rank = ranking_metrics(
    predict_user_based,
    test_df,
    K_VALUES,
    n_users=RANKING_N_USERS,
    max_candidates=RANKING_MAX_CANDIDATES,
)
ibcf_rank = ranking_metrics(
    predict_item_based,
    test_df,
    K_VALUES,
    n_users=RANKING_N_USERS,
    max_candidates=RANKING_MAX_CANDIDATES,
)

svd_rank = ranking_metrics(
    predict_svd, test_df, K_VALUES, n_users=RANKING_N_USERS, max_candidates=RANKING_MAX_CANDIDATES
)
print("Computing ranking metrics for Two-Tower BPR (tracks) …")
two_tower_rank = ranking_metrics(
    predict_two_tower_bpr,
    test_df,
    K_VALUES,
    n_users=RANKING_N_USERS,
    max_candidates=RANKING_MAX_CANDIDATES,
)
print(f"\n{'Model':<24}", end="")
for k in K_VALUES:
    print(f"  P@{k}   R@{k}  NDCG@{k}", end="")
print()
print("-" * 96)

for name, metrics in [
    ("UserBased CF", ubcf_rank),
    ("ItemBased CF", ibcf_rank),
    ("SVD", svd_rank),
    ("Two-Tower BPR (tracks)", two_tower_rank),
]:
    print(f"{name:<24}", end="")
    for k in K_VALUES:
        m = metrics[f"{k}"]
        print(f"  {m['precision']:.4f} {m['recall']:.4f} {m['ndcg']:.4f}", end="")
    print()

results_rows = []
for name, rank_m, rmse, mae in [
    ("User-Based CF", ubcf_rank, ubcf_rmse, ubcf_mae),
    ("Item-Based CF", ibcf_rank, ibcf_rmse, ibcf_mae),
    ("SVD", svd_rank, svd_rmse, svd_mae),
    ("Two-Tower BPR (tracks)", two_tower_rank, two_tower_rmse, two_tower_mae),
]:
    row = {"Model": name, "RMSE": round(rmse, 4), "MAE": round(mae, 4)}
    for k in K_VALUES:
```

```
        row[f"P@{k}"] = round(rank_m[f"{k}"]["precision"], 4)
        row[f"R@{k}"] = round(rank_m[f"{k}"]["recall"], 4)
        row[f"NDCG@{k}"] = round(rank_m[f"{k}"]["ndcg"], 4)
    results_rows.append(row)

results_df = pd.DataFrame(results_rows).set_index("Model")
print("\nConsolidated Results:\n", results_df.to_string())
```

```
Computing ranking metrics …
Computing ranking metrics for Two-Tower BPR (tracks) …

Model                     P@5   R@5  NDCG@5  P@10   R@10  NDCG@10  P@20   R@20  NDCG@20
---------------------------------------------------------------------------------------------
UserBased CF              0.0080 0.0014 0.0200  0.0080 0.0034 0.0342  0.0140 0.0122 0.0748
ItemBased CF              0.0080 0.0020 0.0172  0.0120 0.0066 0.0430  0.0220 0.0189 0.0927
SVD                       0.0640 0.0123 0.1453  0.0400 0.0171 0.1601  0.0240 0.0252 0.1601
Two-Tower BPR (tracks)    0.0880 0.0225 0.2211  0.0600 0.0290 0.2341  0.0440 0.0422 0.2651

Consolidated Results:
                           RMSE     MAE    P@5     R@5  NDCG@5   P@10    R@10  NDCG@10   P@20    R@20  NDCG@20
Model
User-Based CF           2.8491  1.2910  0.008  0.0014  0.0200  0.008  0.0034   0.0342  0.014  0.0122   0.0748
Item-Based CF           2.7658  1.1479  0.008  0.0020  0.0172  0.012  0.0066   0.0430  0.022  0.0189   0.0927
SVD                     2.8166  1.2701  0.064  0.0123  0.1453  0.040  0.0171   0.1601  0.024  0.0252   0.1601
Two-Tower BPR (tracks)  3.2419  2.2100  0.088  0.0225  0.2211  0.060  0.0290   0.2341  0.044  0.0422   0.2651
```

4. Plots and qualitative analysis

Bar and line charts summarize errors and ranking metrics; heatmaps show cosine similarity structure from UBCF/IBCF. **After saving PNGs**, figures are also shown inline for Colab.

```
!pip install -q kaleido
```

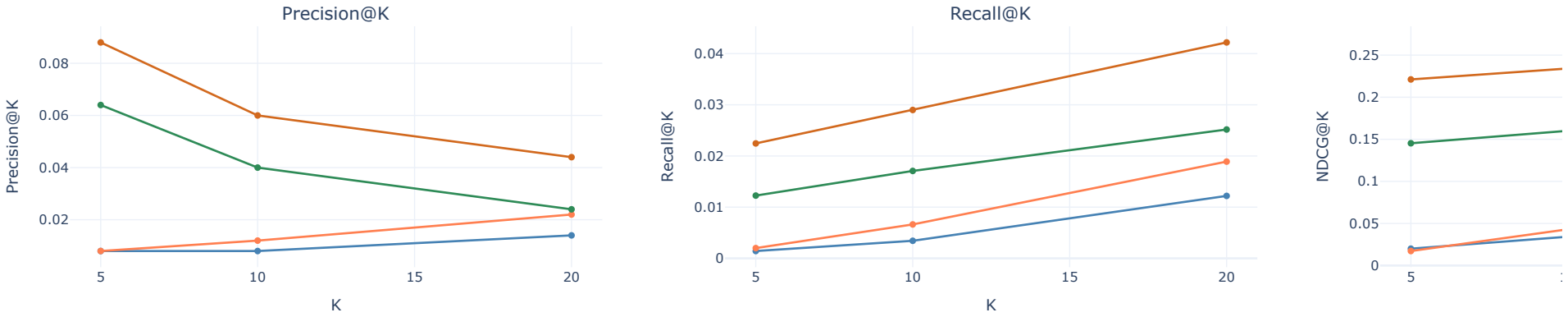
```
# fig_err = go.Figure()
# models = [
#     "User-Based CF",
#     "Item-Based CF",
#     "SVD",
#     "Two-Tower BPR (tracks)",
# ]
# rmse_vals = [
#     ubcf_rmse,
#     ibcf_rmse,
#     svd_rmse,
#     two_tower_rmse,
# ]
# mae_vals = [
#     ubcf_mae,
#     ibcf_mae,
#     svd_mae,
#     two_tower_mae,
# ]
# fig_err.add_trace(go.Bar(name="RMSE", x=models, y=rmse_vals, marker_color="steelblue"))
# fig_err.add_trace(go.Bar(name="MAE", x=models, y=mae_vals, marker_color="coral"))
# fig_err.update_layout(
#     title="Prediction Error: RMSE & MAE by Model",
#     barmode="group",
#     yaxis_title="Error",
#     template="plotly_white",
# )
# # save_plotly_figure(fig_err, "prediction_error_rmse_mae")
# fig_err.show()
```

```
metrics_data = {
    "Precision@K": {
        name: [m[f"{k}"]["precision"] for k in K_VALUES]
        for name, m in [
            ("User-Based CF", ubcf_rank),
            ("Item-Based CF", ibcf_rank),
            ("SVD", svd_rank),
            ("Two-Tower BPR (tracks)", two_tower_rank),
        ]
    },
    "Recall@K": {
        name: [m[f"{k}"]["recall"] for k in K_VALUES]
        for name, m in [
            ("User-Based CF", ubcf_rank),
            ("Item-Based CF", ibcf_rank),
            ("SVD", svd_rank),
            ("Two-Tower BPR (tracks)", two_tower_rank),
        ]
    },
    "NDCG@K": {
        name: [m[f"{k}"]["ndcg"] for k in K_VALUES]
        for name, m in [
            ("User-Based CF", ubcf_rank),
            ("Item-Based CF", ibcf_rank),
            ("SVD", svd_rank),
            ("Two-Tower BPR (tracks)", two_tower_rank),
        ]
    },
}

fig_rank = make_subplots(rows=1, cols=3, subplot_titles=list(metrics_data.keys()))
colors = {
    "User-Based CF": "steelblue",
    "Item-Based CF": "coral",
    "SVD": "seagreen",
    "Two-Tower BPR (tracks)": "chocolate",
}

for col_i, (metric_name, model_dict) in enumerate(metrics_data.items(), start=1):
    for model_name, vals in model_dict.items():
        fig_rank.add_trace(
            go.Scatter(
                x=K_VALUES,
                y=vals,
                mode="lines+markers",
                name=model_name,
                showlegend=(col_i == 1),
                marker_color=colors[model_name],
            ),
            row=1,
            col=col_i,
        )
    fig_rank.update_xaxes(title_text="K", row=1, col=col_i)
    fig_rank.update_yaxes(title_text=metric_name, row=1, col=col_i)

fig_rank.update_layout(title="Ranking Metrics @K by Model", template="plotly_white", height=400)
# save_plotly_figure(fig_rank, "ranking_metrics_at_k")
fig_rank.show()

fig_heat_u = px.imshow(
    user_sim_df,
    title="User-User Cosine Similarity",
    color_continuous_scale="RdBu",
```

```
    zmin=-1,
    zmax=1,
    labels=dict(color="Cosine Sim"),
)
fig_heat_u.update_layout(template="plotly_white")
# save_plotly_figure(fig_heat_u, "user_similarity_heatmap")
fig_heat_u.show()

fig_heat_i = px.imshow(
    item_sim_df,
    title="Item-Item Cosine Similarity",
    color_continuous_scale="RdBu",
    zmin=-1,
    zmax=1,
    labels=dict(color="Cosine Sim"),
)
fig_heat_i.update_layout(template="plotly_white")
# save_plotly_figure(fig_heat_i, "item_similarity_heatmap")
fig_heat_i.show()

ubcf_errors = np.array(ubcf_preds) - np.array(ubcf_actuals)
ibcf_errors = np.array(ibcf_preds) - np.array(ibcf_actuals)
svd_errors = np.array(svd_preds) - np.array(svd_actuals)
two_tower_errors = np.array(two_tower_preds) - np.array(two_tower_actuals)

fig_err_dist = make_subplots(
    rows=2,
    cols=2,
    subplot_titles=[
        "User-Based CF",
        "Item-Based CF",
        "SVD",
        "Two-Tower BPR (tracks)",
    ],
)
fig_err_dist.add_trace(
    go.Histogram(x=ubcf_errors, nbinsx=40, name="User-Based CF", marker_color="steelblue", opacity=0.8),
    row=1,
    col=1,
)
fig_err_dist.add_trace(
    go.Histogram(x=ibcf_errors, nbinsx=40, name="Item-Based CF", marker_color="coral", opacity=0.8),
    row=1,
    col=2,
)
fig_err_dist.add_trace(
    go.Histogram(x=svd_errors, nbinsx=40, name="SVD", marker_color="seagreen", opacity=0.8),
    row=2,
    col=1,
)
fig_err_dist.add_trace(
    go.Histogram(
        x=two_tower_errors, nbinsx=40, name="Two-Tower BPR (tracks)", marker_color="chocolate", opacity=0.8
    ),
    row=2,
    col=2,
)
for r, c in [(1, 1), (1, 2), (2, 1), (2, 2)]:
    fig_err_dist.update_xaxes(title_text="Predicted − Actual", row=r, col=c)
fig_err_dist.update_yaxes(title_text="Count", row=1, col=1)
fig_err_dist.update_layout(
    title="Prediction Error Distributions Across Models",
    template="plotly_white",
    height=520,
```

```
        showlegend=True,
    )
    # save_plotly_figure(fig_err_dist, "prediction_error_distributions")
    fig_err_dist.show()
```

Ranking Metrics @K by Model



User-User Cosine Similarity